

Supplemental Material

EVMBT: A Binary Translation Scheme for Upgrading EVM Smart Contracts to WASM

1 Fixing Performance

The benchmark in Table 1 contains 36 samples, including #15 IB, #10 iRE, #2 cRE, #2 OR, #7 US. We create a testing script for each sample to validate the hardness effectiveness and the introduced overhead. We record the size of patched WASM contracts (in KB) and the increased percentage of the original one.

2 Artifact

Artifact is available at <https://figshare.com/s/ee6d1d9b7fe2c7bcf6af>.

Apart from the standalone compiler version, we also integrate EVMBT with a WASM interpreter as a shared library which the Eth2 node like *geth* can load to replace EVM. The given EVM smart contract is compiled into an LLVM function named *main()*. For EEI compatibility, we declare the EEI function headers in LLVM ASSEMBLY, enabling *main()* to interact with the Ethereum environment by invoking EEI methods. In particular, the return values of EEI functions are converted to little-endianness to conform to the WASM requirements. Once the LLVM ASSEMBLY is established, the LLVM backend in clang [2] generates a WASM object from it. During the linking stage, *main()* and EEI functions are declared as external interfaces exposing to the WASM. This is critical for developers to invoke WASM contracts using the existing toolkit, i.e., Web3.js. **Platform.** EVMBT should run well on any Linux server. Here is our experimental environment. Improving Hera to be available in the industry deserves another paper. For experimental testing, we configured Hera to return predefined constants when responding to specific EEI APIs introduced from the Ethereum “London” upgrade [4], such as EXTERNALCODEHASH, CREATE2, SELFBALANCE, CHAINID, BASEFEE and COINBASE. To accelerate the experiments, we ensure that all blockchain data remains in RAM to minimize I/O latency. Additionally, instead of employing Proof-of-Work, which can be time-consuming due to mining, we utilized Proof-of-Authority consensus mechanism.

System: Ubuntu 18.04 CPU: Intel I7 RAM: 64 GB Dependencies

Name Version git Latest CC gcc-7 CXX g++-7 cmake 3.20.0 LLVM 10.0

Build. We build EVMBT to a Linux x64 ELF. First we setup the build environment.

```
1 $ sudo apt-get update && apt-get install gcc-7
   apt-get install g++-7 -y
2 $ export CC=/usr/bin/gcc-7
3 $ export CXX=/usr/bin/g++-7
```

```
4 $ sudo ln -s /usr/bin/python2.7 /usr/bin/python
   # link % python if your are at ubuntu 20.04
5 $ git clone <git-repo> project && cd project #
   download the artifact
6 $ ./script/install_cmake.sh # install cmake
   3.20.0
```

We can obtain the binary of EVMBT.

```
1 $ wget https://github.com/ethsema/ethsema-paper/
   releases/download/review/standalone-evmtrans
```

We build a testnet with a geth node (commit-0x6c61eba), which uses Hera (commit-0xa396507) as the eWASM VM and maintains compatibility to EVM.

```
1 $ docker build -t localhost/client-go:ewasm .
2 $ ./scripts/ewasm.sh
```

Once the environment is established, we can run the below code to test EVMBT.

```
1 $ python -m runtimeTest.reentrancy --overwrite
   --langArgs="--check-reentrancy"
```

3 B2 Benchmark

Table 2 shows the benchmark B2, including 14 different smart contracts [5] written in Solidity. They can be grouped into four categories according to their functionalities. *simple operation* includes eight smart contracts, in which each one differs only in key the repeated opcodes. The smart contracts in *arithmetic*, *EEI*, and *hashing* groups come from the most widely used blockchain test sets: Ethereum Consensus Tests [1] and Blockbench [3].

Smart contracts in group *simple operation* repeat a specific operation x times, which by default is set to be 10,000. Smart contracts in group *arithmetic* run arithmetic operations which are more complicated than those in *simple operation*, such as Fibonacci sequence calculation and matrix multiplication. x is set to be 10,000 for *fib*, and 100 for *matrix* and *warshall* due to the stack limit of WASM execution engines. Smart contracts in group *EEI* read the status of current Ethereum and repeat 10,000 times. Smart contracts in group *hashing* compute hash values using different hashing algorithms. The *keccak256* and *sha256* calculate hash values. Since EVM does not support floating points, all smart contracts are implemented in 256-bit integers.

References

[1] 2020. Ethereum Consensus Tests. <https://github.com/ethereum/tests>.

Table 1. Fixing results on the SmartBugs samples.

MD5	e3c533a2	261f39d0	8f51fb8f	c5db64b8	2cd9935d	60242eb2	7dad2111	f01e8d9b	2797f015
Bug	IB	IB	IB	IB	IB	IB	IB	IB	IB
Size(+%)	7.88 (8.63%)	16.86 (5.96%)	7.89 (8.56%)	8.78 (7.68%)	9.57 (7.01%)	8.70 (12.06%)	6.86 (9.70%)	7.46 (9.04%)	7.88 (8.58%)
Gas (+%)	26618 (0.00%)	26618 (0.00%)	26682 (0.00%)	27562 (0.00%)	27362 (0.00%)	26682 (0.00%)	27544 (0.00%)	21682 (0.00%)	26680 (0.00%)
MD5	a8ad70d3	275680f9	d2cbd0ab	071c89cc	bc5e54fc	1a4cfdc0	68bee99e	de5e14ff	dbce791c
Bug	IB	IB	IB	IB	IB	IB	iRE	iRE	iRE
Size(+%)	6.62 (10.37%)	14.08 (7.52%)	21.67 (10.89%)	21.40 (10.32%)	16.45 (9.84%)	19.19 (9.56%)	13.85 (4.77%)	20.45 (4.60%)	10.03 (5.30%)
Gas (+%)	41680 (0.00%)	34530 (0.00%)	48645 (0.00%)	45274 (0.00%)	35812 (0.00%)	42476 (0.00%)	57084 (21.76%)	109227 (22.97%)	58755 (21.01%)
MD5	dd955dd4	8941f9c8	6f5977ac	fab49dda	8cb99c64	e2087c00	5408b325	f5ba5fbc	152e96e0
Bug	iRE	iRE	iRE	iRE	iRE	iRE	iRE	cRE	cRE
Size(+%)	10.29 (4.15%)	14.41 (3.78%)	11.01 (4.99%)	14.59 (4.10%)	10.81 (3.66%)	11.81 (3.65%)	7.45 (5.06%)	17.40 (3.77%)	15.93 (3.36%)
Gas (+%)	77535 (24.58%)	63050 (32.04%)	62248 (32.59%)	63050 (32.04%)	155795 (7.60%)	75460 (15.63%)	52750 (23.97%)	72034 (39.51%)	75456 (25.43%)
MD5	55a59741	be206fd7	840e9340	fbf2d2b1	38851b4d	590b2bd7	1894fee3	08c18705	1a66f54e
Bug	OR	OR	US	US	US	US	US	US	US
Size(+%)	11 (-0.03%)	11 (-0.04%)	7.71 (0.00%)	6.96 (0.00%)	15.59 (0.00%)	111.20 (0.0%)	90.80 (0.00%)	9.05 (0.00%)	12.58 (0.00%)
Gas (+%)	42898 (0.00%)	23673 (0.00%)	24572 (0.00%)	31310 (0.00%)	33954 (0.00%)	38025 (0.00%)	152539 (0.00%)	25560 (0.00%)	23822 (0.00%)

Table 2. An overview of the synthesis benchmark (B2)

	Contract Name	Description
Simple Operation	add	repeat ADD operation x times
	sub	repeat SUB operation x times
	shr	repeat SHR operation x times
	shl	repeat SHL operation x times
	div	repeat DIV operation x times
	mod	repeat MOD operation x times
	power	repeat MUL operation x times
	exp	repeat EXP operation x times
Arithmetic	fib	calculate the x th term of the Fibonacci sequence
	matrix	do matrix multiplication x times
	warshall	find the shortest paths in a graph with x nodes using Floyd-Warshall algorithm
EEl	builtin	load Ethereum states with all EEI methods
Hashing	keccak256	Ethereum hashing function, keccak256
	sha256	SHA-1, producing a 32-byte hash value

- [2] 2022. The Github repository of the LLVM WebAssembly backend. <https://github.com/llvm/llvm-project/tree/main/llvm/lib/Target/WebAssembly>
- [3] Tien Tuan Anh Dinh, Ji Wang, Gang Chen, Rui Liu, Beng Chin Ooi, and Kian-Lee Tan. 2017. Blockbench: A framework for analyzing private blockchains. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1085–1100.
- [4] et. al. V. Buterin, E. Conner. 2018. EIP-1559: Fee market change for ETH 1.0 chain. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-1559>.
- [5] S. Zheng, H. Wang, L. Wu, G. Huang, and X. Liu. 2020. VM Matters: A Comparison of WASM VMs and EVMs in the Performance of Blockchain Smart Contracts. *arXiv preprint arXiv:2012.01032* (2020).